application with and without your cache, along with various caching approaches, will help fine-tune your implementation. In summary, the shorter the TTL, the higher the load on the underlying data store. The higher the TTL, the more out of sync the data gets.

The write-through strategy (caching data up front) is implemented differently to tackle the synchronization problem, as shown here:

1. The application writes data to the data store and the cache (or the cache is filled asynchronously, for example, by a background job, an AWS Lambda function, or the application).
2. If the application wants to read the data at a later time, it makes a request to the caching layer, which always contains the latest data.
3. The value is returned to the client.

This strategy also comes with a problem. What if the cache is not big enough to contain all your data? Caches are in memory, and your data store's disk capacity is usually larger than your cache's memory capacity. When your cache exceeds the available memory, it will evict data or stop accepting new data. In both situations, the application stops working. A global leaderboard will most likely fit into the cache. Imagine that a leaderboard is 4 KB in size and the cache has a capacity of 1 GB (1,048,576 KB). But what about introducing leaderboards per team? You can only store 262,144 (1,048,576/4) leaderboards, so if you have more teams than that, you will run into a capacity issue. Figure 11.2 compares the two caching strategies.